Promises & Async JS

The Ultimate Beginner's Guide to Fetch, Async, and Await

1. The Big Problem: Why do we need this?

Imagine you are a chef in a kitchen (JavaScript). You are the **only** chef (JavaScript is "single-threaded"). You get an order to bake a cake, which takes 30 minutes in the oven.

If you stand in front of the oven doing nothing for 30 minutes, all the other customers will get angry. The entire restaurant (your website) freezes.

Instead, you put the cake in the oven, set a timer, and go chop vegetables for the next order. When the timer rings, you go back to the cake. This is called **Asynchronous JavaScript**. It allows JavaScript to start a long task (like fetching data from the internet), move on to other things, and come back when the task is done.

2. What is a Promise? (The Restaurant Buzzer)

A **Promise** is how JavaScript handles these long tasks.

Imagine you go to a busy restaurant. The host gives you a glowing buzzer. That buzzer is a **Promise**. It doesn't give you food immediately, but it is a *guarantee* that eventually, you will either get a table, or they will tell you they are closed.

A Promise has exactly three states:

- **Pending:** You are holding the buzzer. It hasn't buzzed yet. The data is still loading.
- **Fulfilled (Resolved):** The buzzer flashes! You got your table (The data successfully arrived).
- **Rejected:** The host comes over and says the kitchen caught on fire. (The internet disconnected or the task failed).

3. Creating a Promise from Scratch

To create a promise, we use `new Promise`. It takes two special tools: `resolve` (the success button) and `reject` (the failure button).

```
// We create a Promise and store it in a variable const myBuzzer = new Promise(function(resolve, reject) { // setTimeout simulates a task that takes 2 seconds (2000ms) setTimeout(function() { let kitchenOnFire = false; if (kitchenOnFire === false) { // Success! We press the resolve button and pass the data. resolve("Your table is ready!"); } else { // Failure! We press the reject button and pass an error message. reject("Sorry, the kitchen is on fire."); } }, 2000); });
```

4. Consuming a Promise (.then and .catch)

Now that we have our `myBuzzer` promise, we need to tell JavaScript what to do when it finally resolves or rejects. We do this using `.then()` for success and `.catch()` for errors.

```
myBuzzer .then(function(message) { // This runs ONLY if resolve() was called. // 'message' becomes "Your table is ready!" console.log("Hooray: " + message); }) .catch(function(error) { // This runs ONLY if reject() was called. // 'error' becomes "Sorry, the kitchen is on fire." console.log("Oh no: " + error); }) .finally(function() { // This runs ALWAYS, no matter what happened. console.log("I am leaving the restaurant now."); });
```

> 💡 **Tutor's Note: The Chain**
>
> Notice how we chain them together: `myBuzzer.then().catch().finally()`. JavaScript will wait the 2 seconds, check if it was resolved or rejected, and jump to the correct block automatically.

5. Enter `fetch`: The Ultimate Built-in Promise

In the real world, you rarely write `new Promise` yourself. Usually, you just want to get data from a URL (like a list of users or weather data).

JavaScript gives us a built-in tool called `fetch()`.
`fetch()` is basically a robot that goes to a URL, grabs the data, and comes back. Because this takes time, **fetch automatically returns a Promise!** You don't have to build it; you just consume it.

```
fetch('https://api.github.com/users/hiteshchoudhary') .then(function(response) { // Step 1: We got a response from the internet! // But the internet only speaks in pure text (Strings). // We must translate this text back into a JavaScript Object using .json() return response.json(); }) .then(function(data) { // Step 2: Now we have the actual translated data! console.log("Followers: " + data.followers); }) .catch(function(error) { // Step 3: Catch any network errors (like no wifi) console.log("Network error: ", error); });
```

6. The JSON Translation Step (Crucial!)

Why are there *two* `.then()` blocks in the fetch code above? This confuses many beginners.

When `fetch` gets data from the internet, it arrives as a massive, flat string of text. JavaScript cannot read a string like an object. `response.json()` is a translator. It takes the text and converts it into a usable JavaScript object (JSON).

However, translating a massive file *also* takes time! So, `response.json()` returns a *second* Promise. That is why we need the second `.then(function(data))` to wait for the translation to finish.

7. The Modern Way: async and await

Using `.then()` is great, but if you have a lot of them, your code starts indenting endlessly to the right (known as "Callback Hell").

In 2017, JavaScript introduced `async` and `await`. It is **not** a new concept. It is just a prettier, easier-to-read way of writing Promises. It makes asynchronous code look like normal, line-by-line synchronous code.

- **await:** Think of this as a "pause button". When you put `await` in front of a Promise (like `fetch`), JavaScript hits pause on that specific function and waits for the data to arrive before moving to the next line.
- **async:** JavaScript has a strict rule: You are **not allowed** to use the `await` pause button unless you label the function it lives in with the word `async`. They are best friends; one cannot exist without the other.

```
      // 1. We MUST label the function as 'async' async function getUserData() { // 2.
We use try/catch to handle errors (instead of .catch()) try { // 3. We use 'await' to
hit pause until the fetch gets back! console.log("1. Sending robot to the
internet..."); const response = await
fetch('https://api.github.com/users/hiteshchoudhary'); // 4. We use 'await' again to
hit pause while it translates the text console.log("2. Robot returned! Translating text
to JSON..."); const data = await response.json(); // 5. Finally, use the data!
console.log("3. Done! Followers:", data.followers); } catch (error) {
console.log("Oops, something broke: ", error); } } // Call the function getUserData();
```

⚠️ Why do we need try...catch?

When we used `.then()`, we had `.catch()` to handle errors. But `await` doesn't have a `.catch()` attached to it. If the Wi-Fi drops and the fetch fails, `await` will completely crash your program. By wrapping our code in a standard JavaScript `try { } catch(error) { }` block, we give the program a safe place to land if the `await` line explodes.

8. Summary Comparison

Which one should you use? Both do the exact same thing behind the scenes. `async/await` is usually preferred today because it is much easier to read, especially when doing multiple fetches in a row.

`fetch().then()` **(The Old Way)**	`async / await` **(The Modern Way)**
Uses `.then()` and `.catch()` to handle success and failure.	Uses a `try...catch` block to handle errors.
Code executes inside nested callback functions.	Code executes top-to-bottom, looking just like normal synchronous code.
No need to label the surrounding function.	**Must** label the surrounding function with `async`.